

[原创]某FPS外挂网络验证分析与网络验证服务端API的漏洞

 mb_owcppmrg 

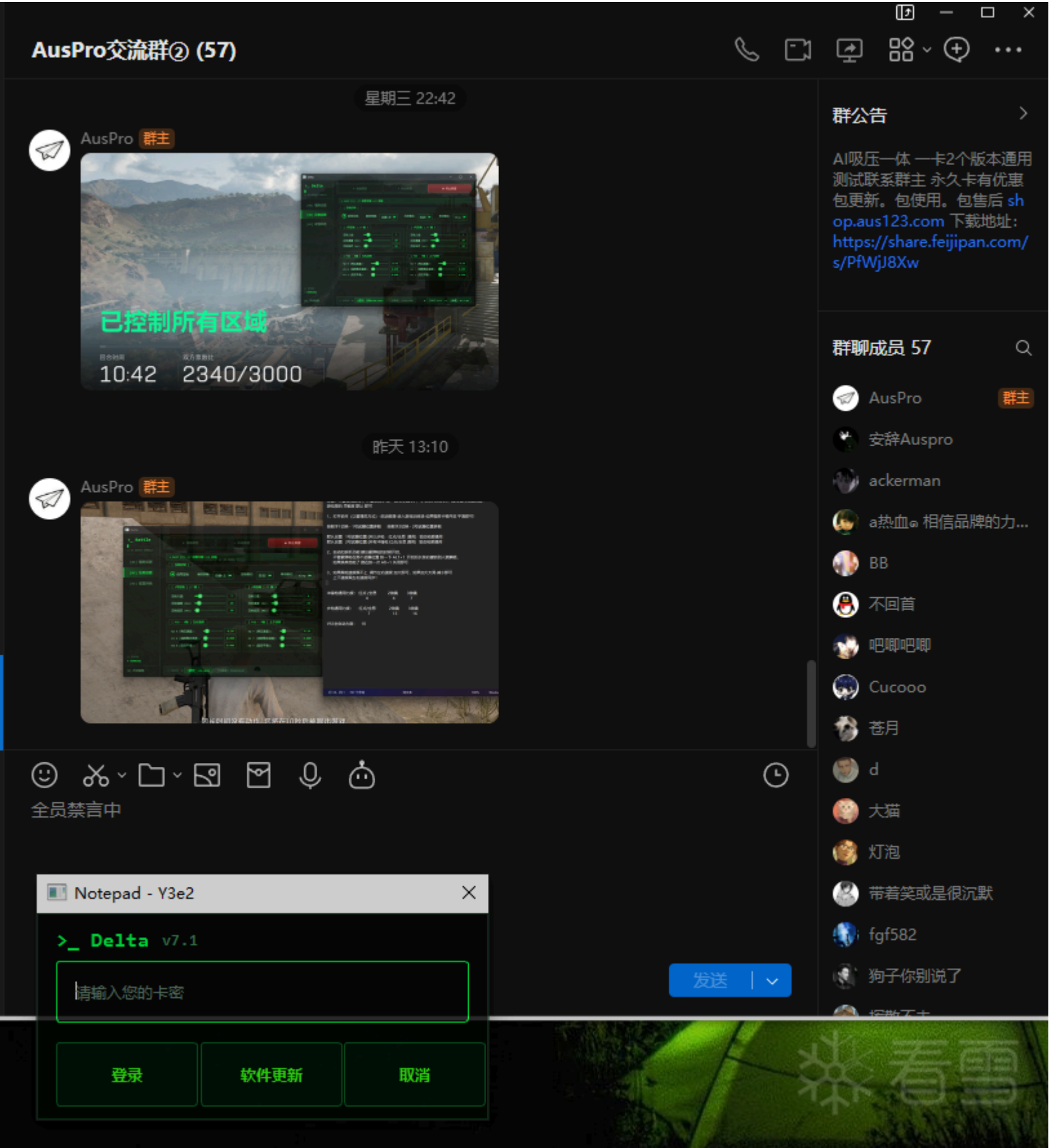
临时 ☆

4天前

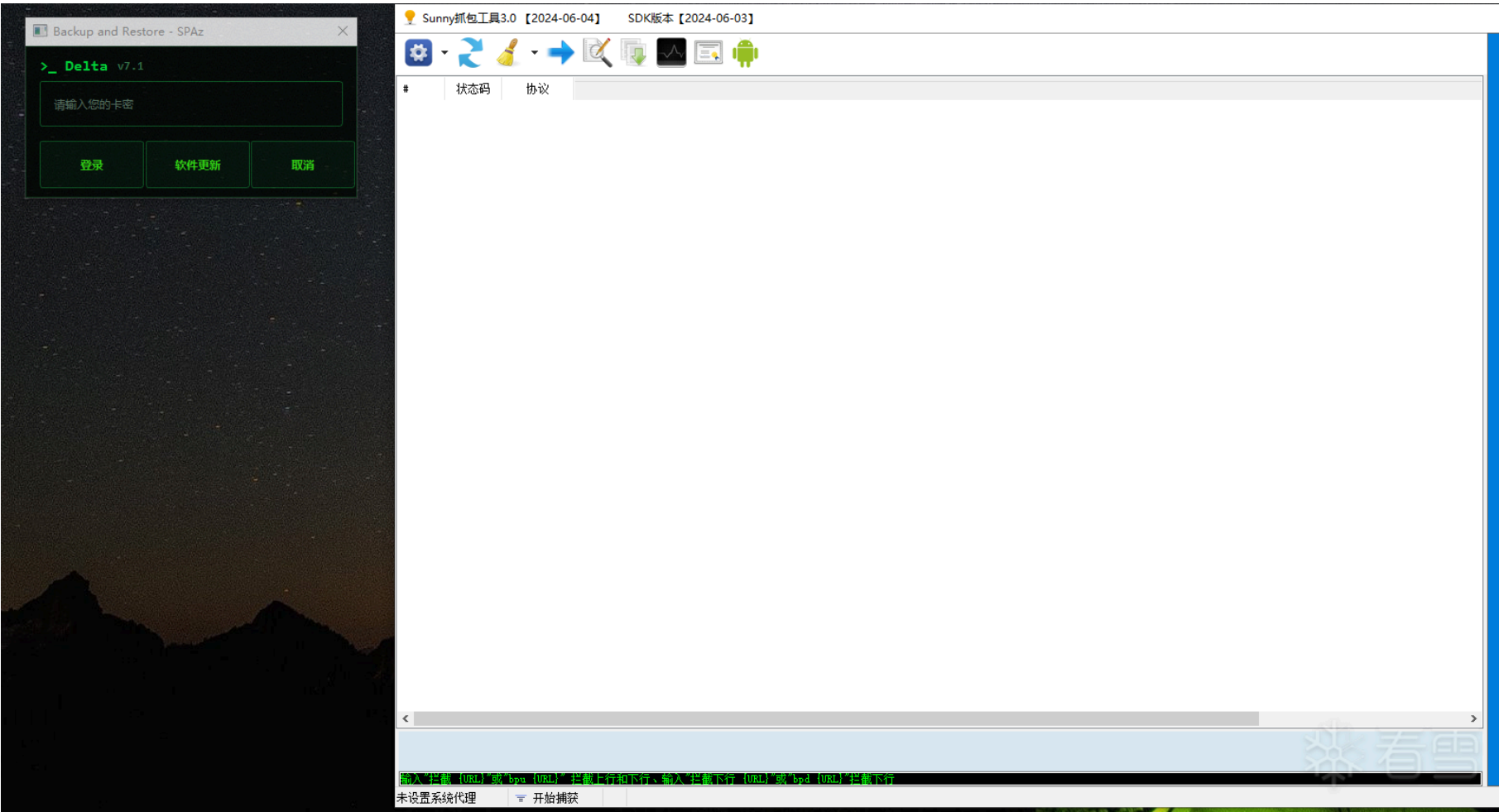
 举报

 218

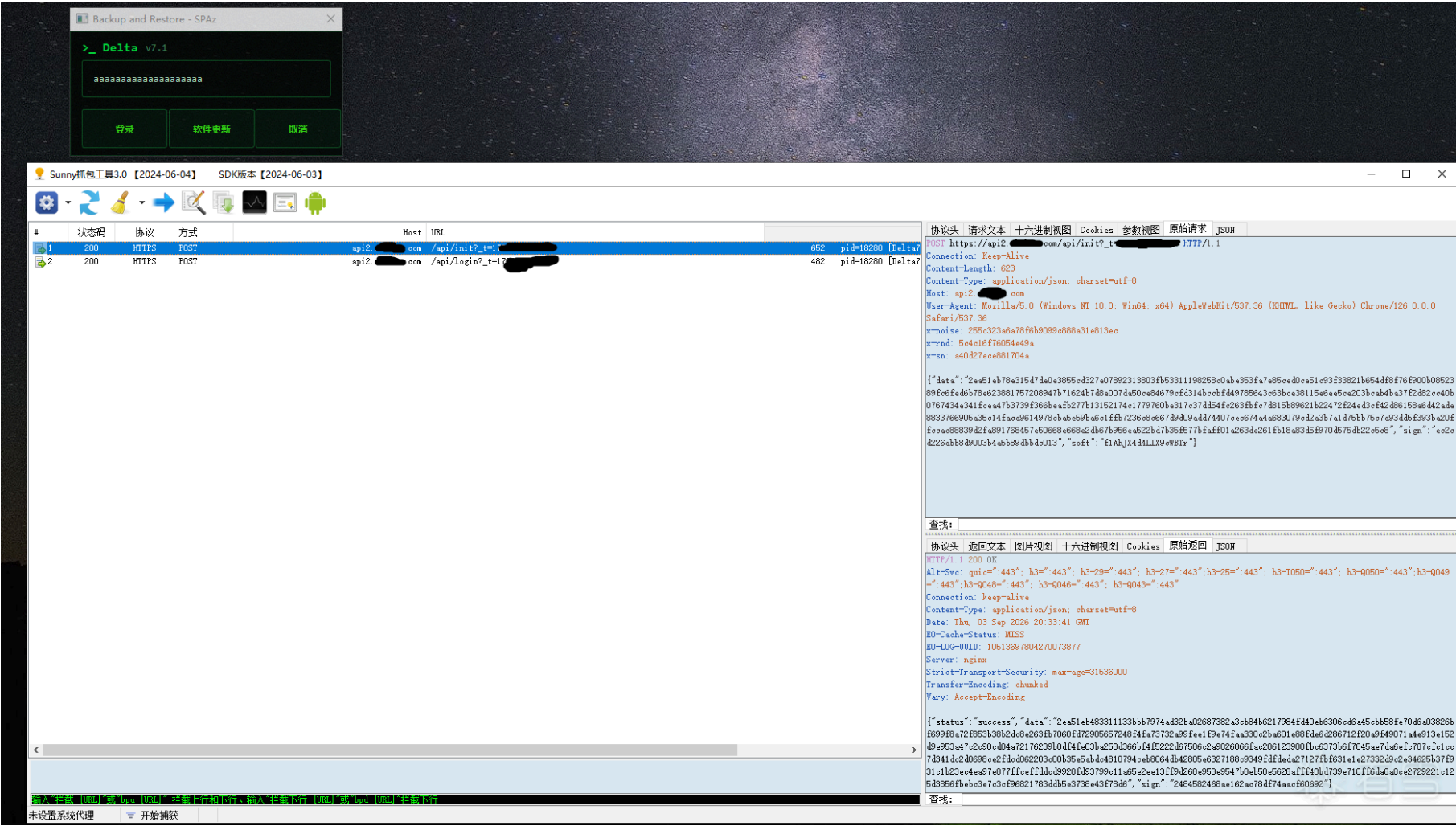
最近偶尔在玩PUBG，屡屡被各种离谱的操作当成路边一条狠狠踹死.....然而在逛B站的时候刷到一个宣称非常NB的“AI自瞄”，什么锁头、压枪、动态预测之类的功能应有尽有.....本篇暂且不讨论它的作弊手法,主要讨论该外挂的网络验证逆向分析(主要用"山寨"的思路)卧底他们的群之后发现有PUBG和三角洲这两个游戏的版本，用的网络验证都是一样的，随便选个吧



抓包看看，软件从双击打开到出现登录界面，期间没有任何网络通信行为



那么就随便输入几个字符点登录看看



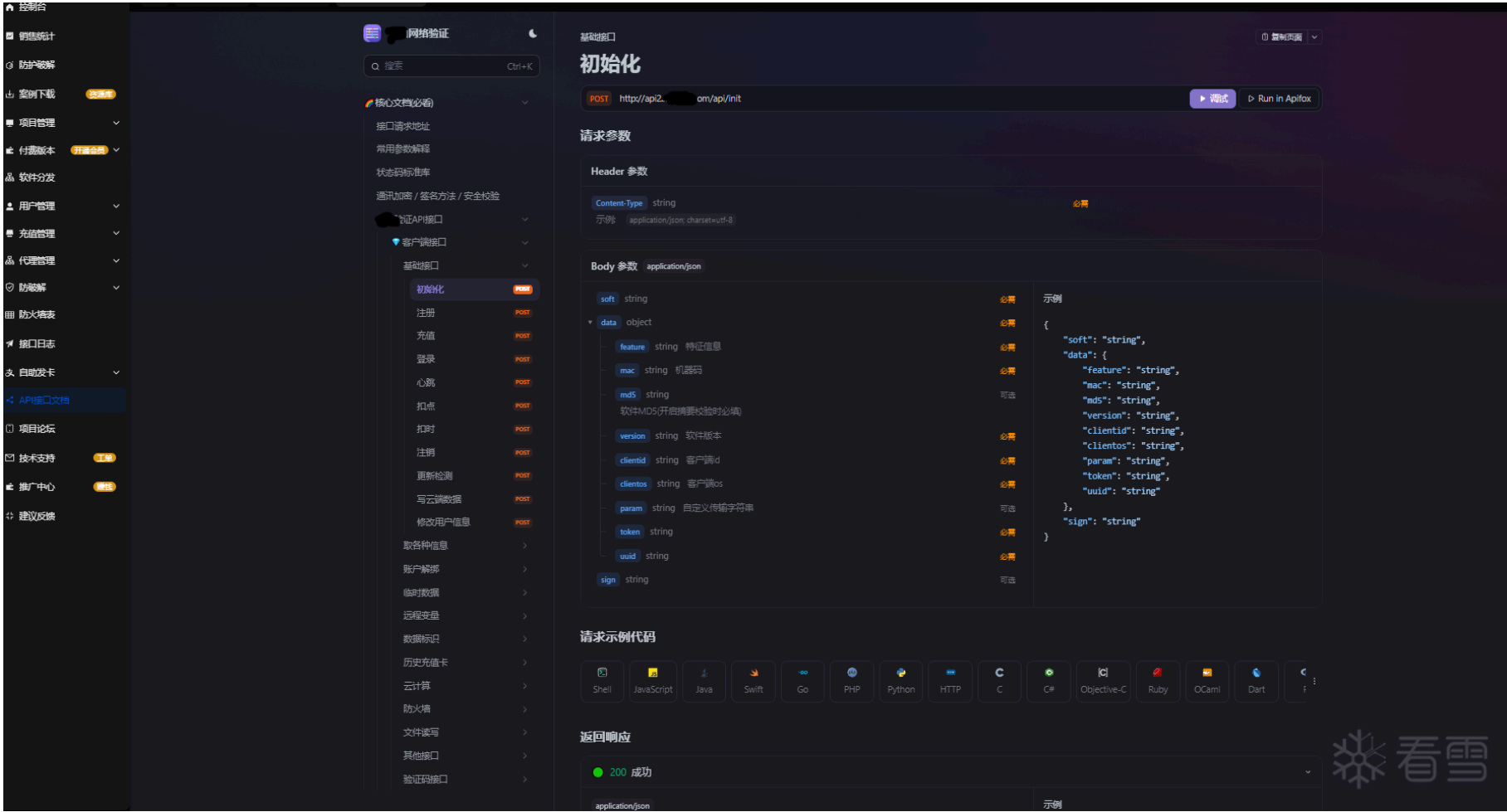
可以看到POST两次,一个init和一个login,后面还带着时间戳,而且每次点登录都会POST一下init和login

这网络验证很经典啊,都不用扫域名什么的,去掉api2,后面的就是官网了..

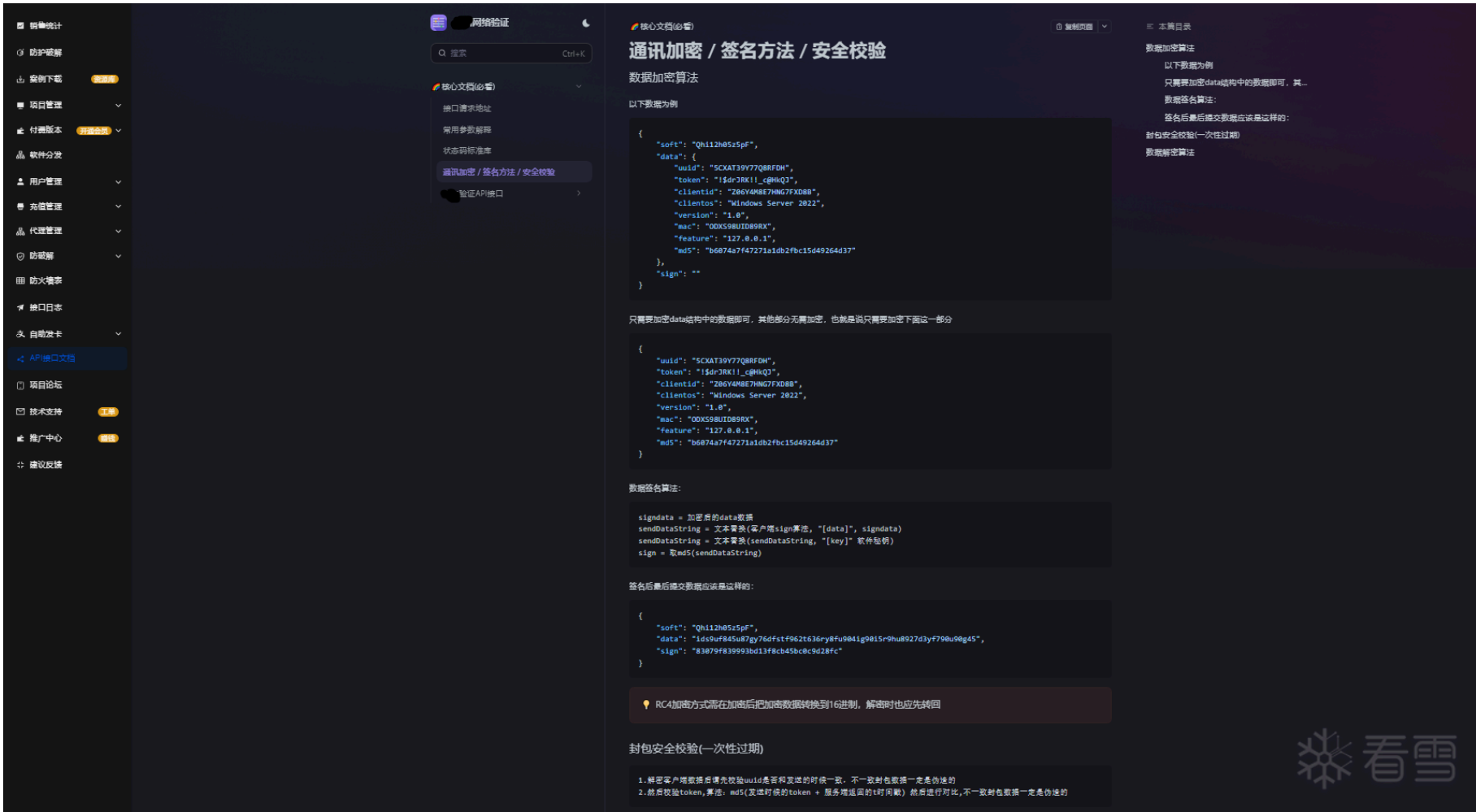
注册是免费的,注册进去看下,进去以后直奔API接口文档去看看



(这话给我看红温了哈哈哈)...可以看到，有三个接口请求地址，这个外挂用的是第一个，也就是api2那个
然后看下大概有哪些接口



再看下通讯加密方面的说明



太经典了，这种我连xdbg都懒得开，一个抓包加一个CE基本上能把所有事情都做了...

这种网络验证的本质很简单:

对于服务器而言，软件识别码是最重要的，我的理解是,在服务器眼里并不是通过你注册这个网络验证时的用户名/UIID之类的东西去分辨软件位的，因为每个用户可能拥有多个软件位，直接引入一个软件识别码，这个软件识别码可能是相对用户唯一，也有可能是相对整个服务器唯一，不过这对我们今天的论题来说不重要，简单来说就是 服务器是通过软件识别码来分辨软件位的

而每一个软件位中的必要参数主要有两个:

- 1、软件密钥
- 2、通讯密钥/密钥对(密钥对是非对称加密用的)

关于软件密钥，这个东西暂时不知道是做什么用的，因为通讯其实要一个通讯密钥就够加解密上下行数据了，不过用我自己的一点经验来看的话，上下行数据中都有sign这个字段，而这个软件密钥这个东西大概率是算sign的一个主要因素(后面证实了这个猜测，是对的)

关于通讯密钥/密钥对:

如果是对称加密，比如AES、DES、RC4这类，很简单，只需要拿到通讯密钥直接加/解密就行了，可以像这篇一样"山寨"，也可以自己做个后台，用相同的算法相同的密钥去加解密，自己随便伪造下行数据发给客户端就行

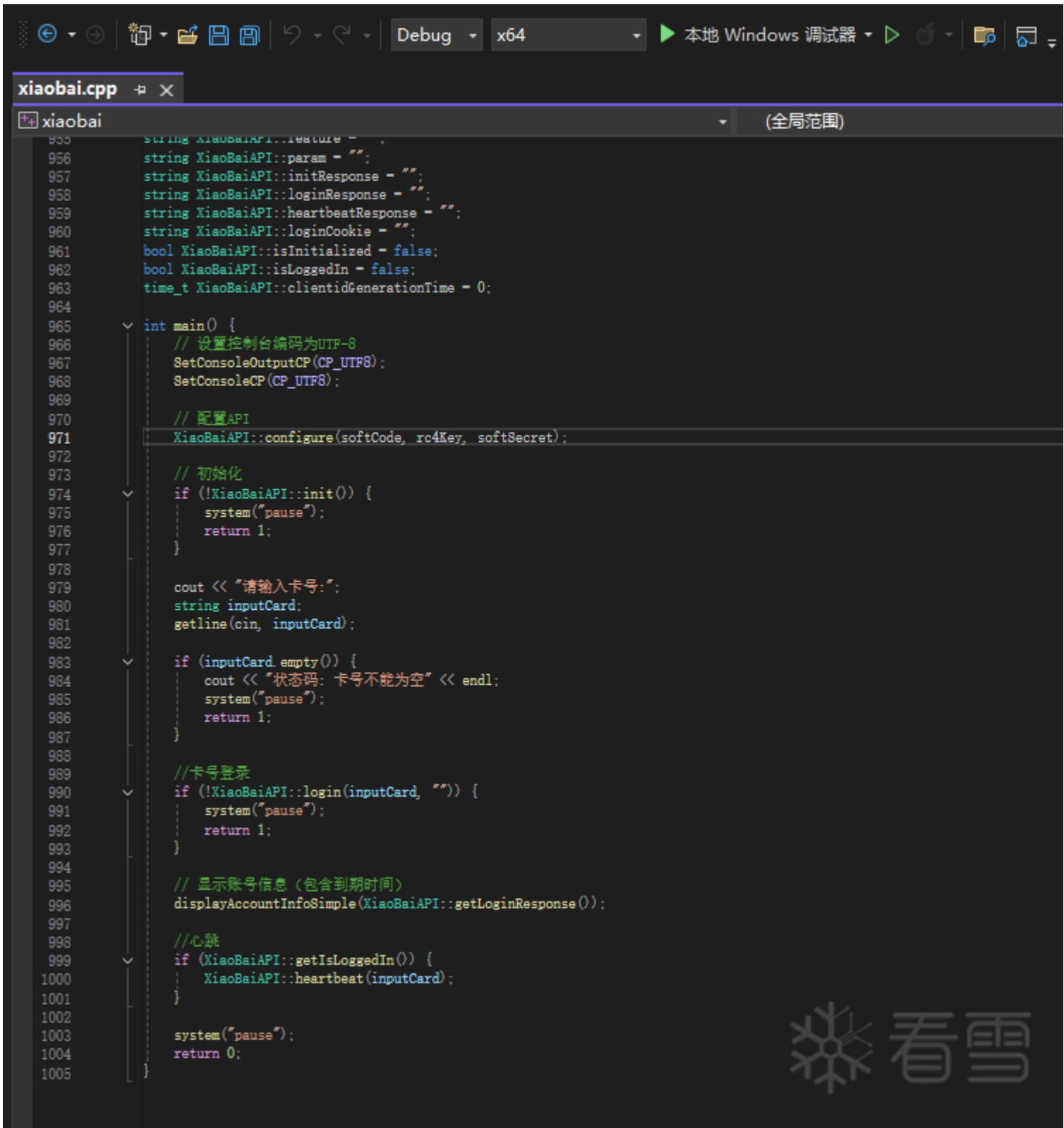
如果是非对称加密，比如RSA这类,其实也不用慌，在对称加密的情况下，一个通讯密钥兼顾加密和解密两件事情，而如果是非对称加密，客户端这边能找到的通讯密钥只能是公钥，而对于RSA这种算法来说，公钥不是秘密，如果采用了非对称算法，通常都是使用ECDH椭圆曲线算法协商一个对称密钥，底层依然用对称加密算法进行加/解密，但通常这个协商出来的对称密钥是一次性的，否则就失去了非对称算法的一部分意义

对于这类网络验证来说，无论是对称还是非对称算法，都救不了它，因为对称密钥可以直接从客户端找通讯密钥，在合适的时机直接改成自己的，或者自己伪造服务端然后客户端直接转向就可以了

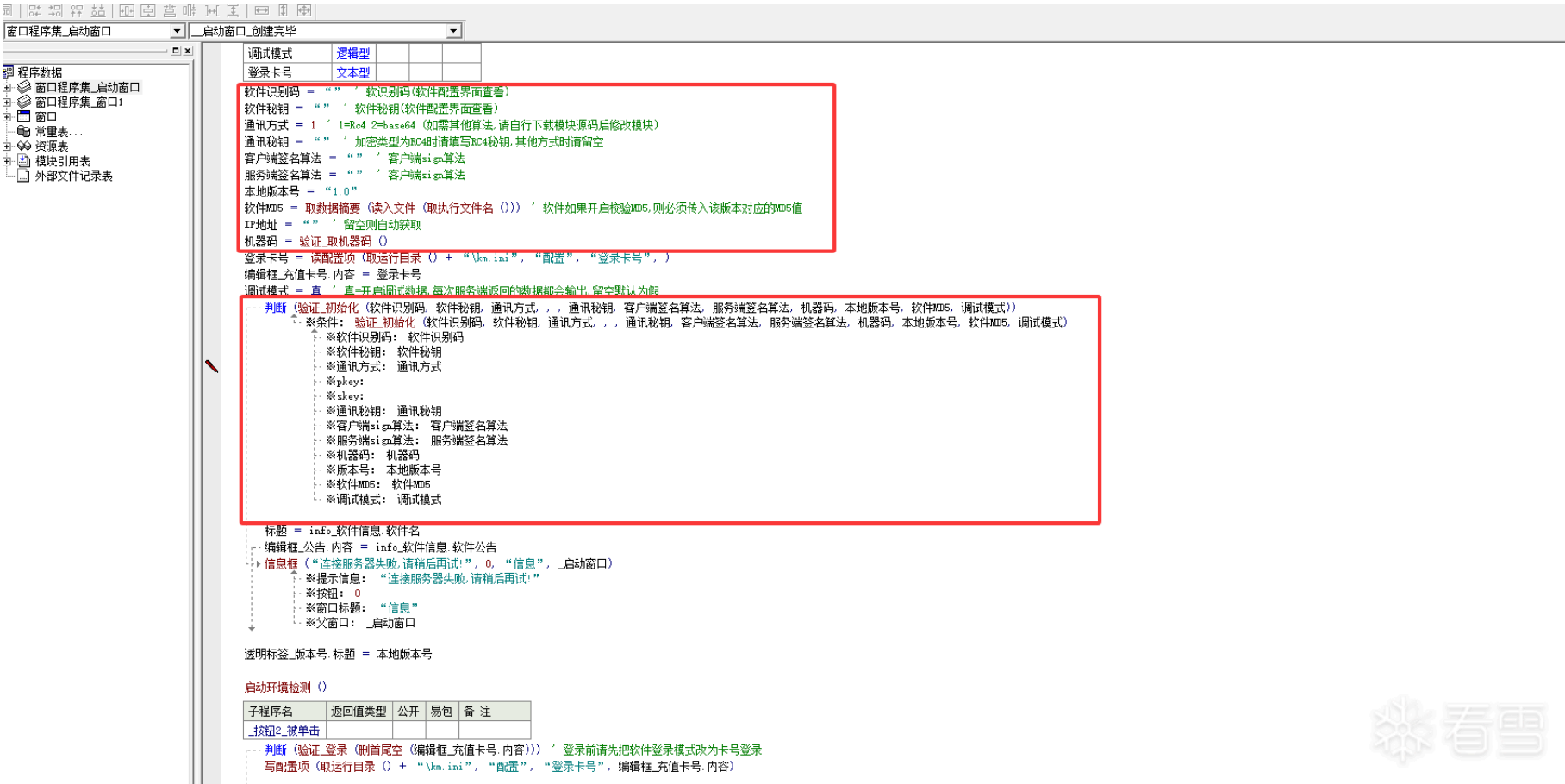
而如果是非对称加密，基本上一样的手法，自己生成密钥对，填入后台，然后修改客户端里的公钥为自己生成的密钥对中的公钥，这样客户端和服务端就能互相听懂对方在说什么了，或者自己写个服务端，然后转向

而现在最重要的就是初始化部分，因为初始化需要很多因素，比如:软件的识别码、软件密钥、通讯密钥、版本号等等，这些关键信息都是初始化函数/接口的必要参数

下载它的demo看看



从上面的截图中可以看到先是调用了 类函数 configure 把softCode也就是软件识别码、rc4Key也就是通讯密钥、softSecret也就是软件密钥传进去了
不过它C的demo貌似写的不全，接口文档里可以看到是支持很多种加密方式的，下载了它的易语言的demo之后看到是全的

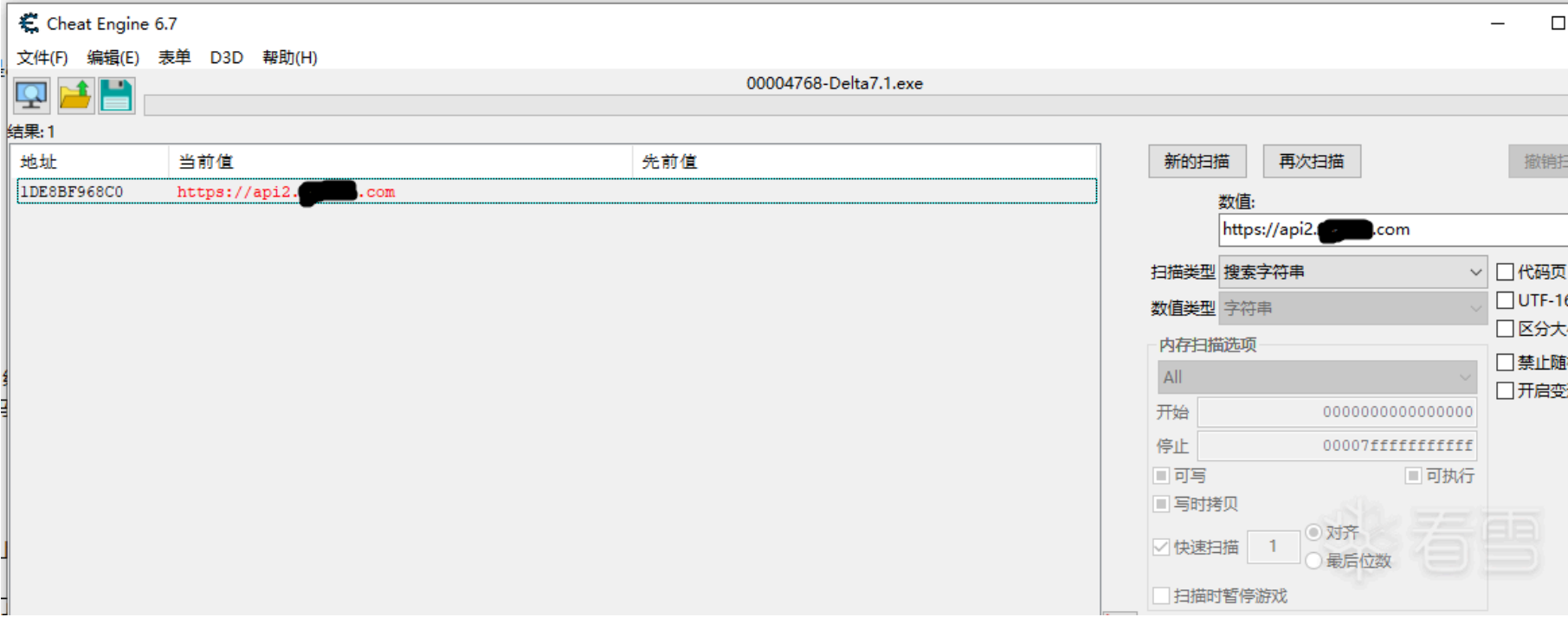


C里面是用类函数初始化的，易语言里面是把这些关键信息全部保存在一个全局变量结构体里面查壳就算了，默认它是VMP...

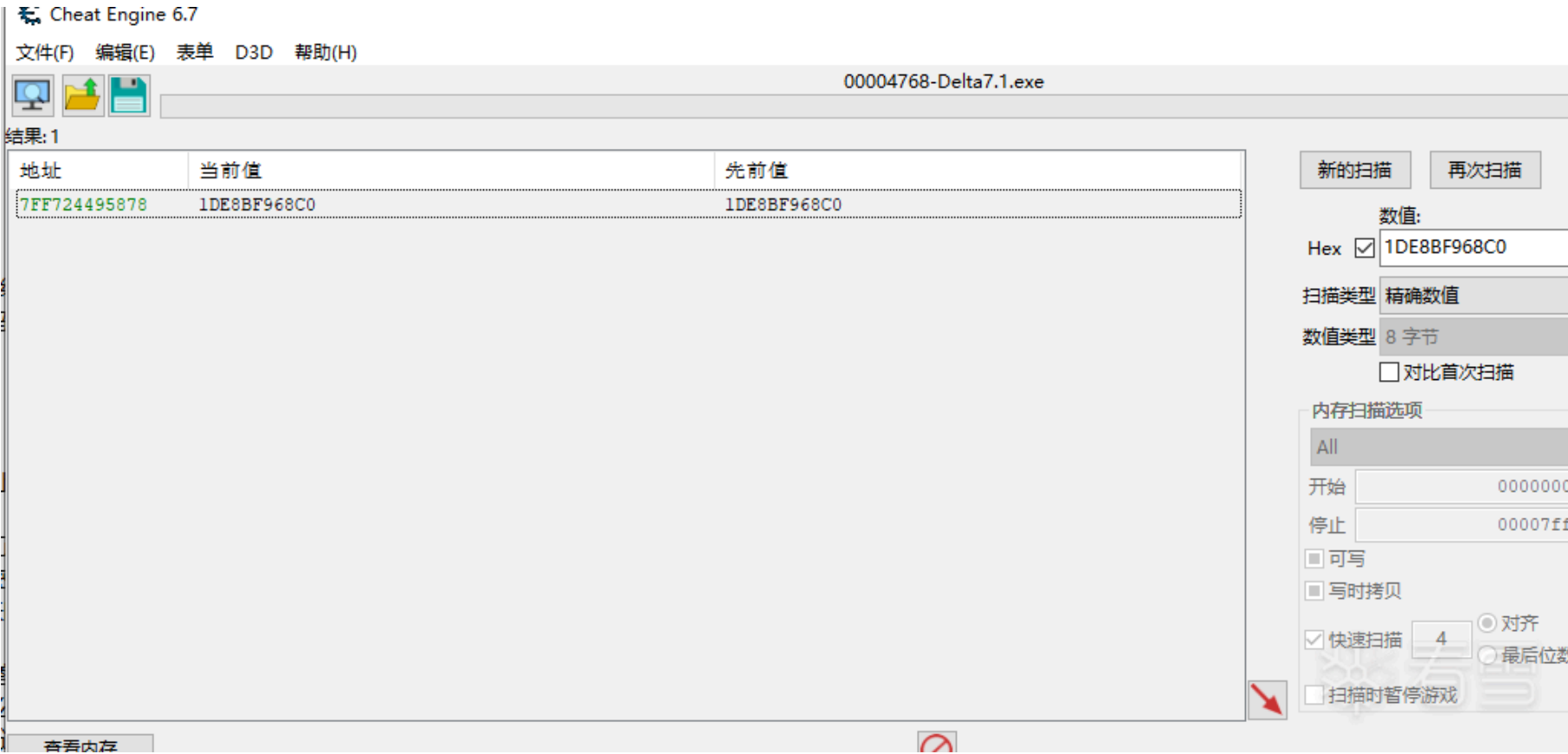
如果是VMP的话，像初始化函数、静态字符串这些基本都是会被VM的，所以想直接找它硬编码的密钥之类的字符串直接改是不行的
这里有两种方法可以尝试：

- 1、无论是类函数还是结构体又或者无论是不是VMP这类强壳，本质上都差不多，是类函数就先找它的类，是结构体就找它的结构体，用已知的信息找，比如版本号、URL这些，事实上也可以用软件识别码去找，因为接口文档中明确写了软件的识别码是不被加密的，而软件识别码会出现在POST的请求体中，找到了类/结构体以后，旁边的内存地址里面就会存放着其他的关键信息
- 2、如果上述方法行不通，就找它的初始化函数，或者说相关函数，而初始化相关的函数一旦被调用，就必然会出现这些关键信息，到时候选个合适的位置挂个VEH就能读写

前面都在讲理论现在开始操作，直接搜域名



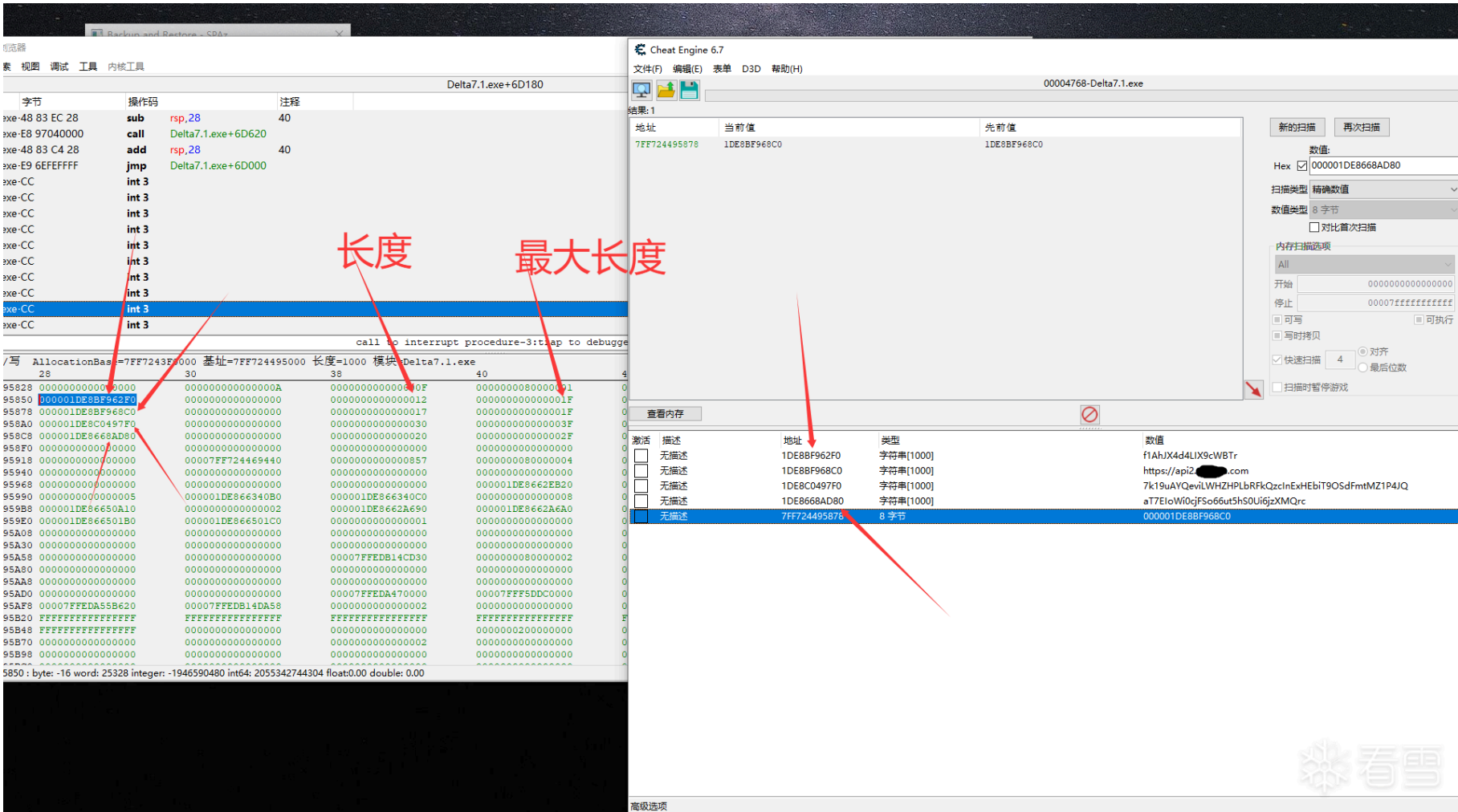
搜出来一个，然后尝试找它的指针



找到一个，然后Ctrl+B去看附近的地址

接下来

是纯正的欧美打法,上去直接全部干出来:



验证一下,不能太武断:

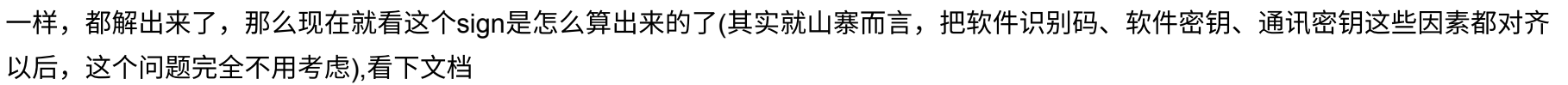
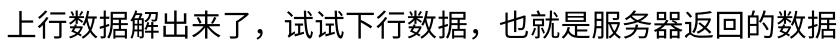
正如拉马努金在梦中得到了女神的启示那样，我们同样敏锐地注意到:

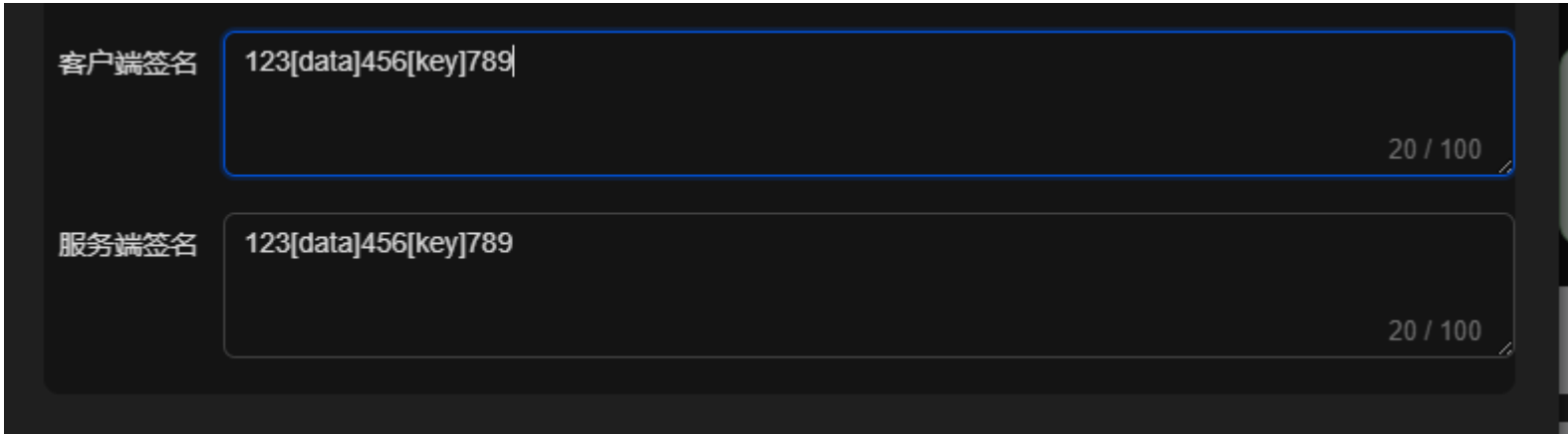
7k19uAYQeviLWHZHPLbRFkQzclnExHEbiT9OSdFmtMZ1P4JQ 这个字符串一共是48字节，所以它显然用的是RC4算法，且这个字符串必定是通讯密钥，至于为什么....别问，问就是女神的启示....开个玩笑，其实AES的密钥长度只能是16、24、32字节，而这个网络验证只支持明文、RC4、Base64编码、RSA、DES、AES，只有RC4符合这个长度了...暂且闭着眼睛认为它用的就是RC4算法且通讯密钥就是这个...

f1AhJX4d4LIX9cWBTr 这个字符串是18字节,整个网络验证里只有识别码是18字节，暂且认为它就是软件识别码

aT7EIoWi0cjFS066ut5hS0Ui6jzXMQRc 一共是32字节，这就有点悬念了，毕竟如果是AES算法，那AES的密钥长度也有可能是这个，不过因为软件密钥也一样固定是这个长度，所以暂且认为它就是软件密钥....

现在验证一下算法和密钥对不对得上





验证的开发者还真是个小机灵鬼，这部分其实是个双向验证，客户端签名验证 和 服务器签名验证，用的是:固定字符串+密文请求体+固定字符串+软件密钥+固定字符串；这样的方式算md5,不过没什么用啊....

也就是说:

服务器验证sign字段的方式是:123+密文的上行数据+456+软件密钥+789组成一个字符串，然后算它的MD5

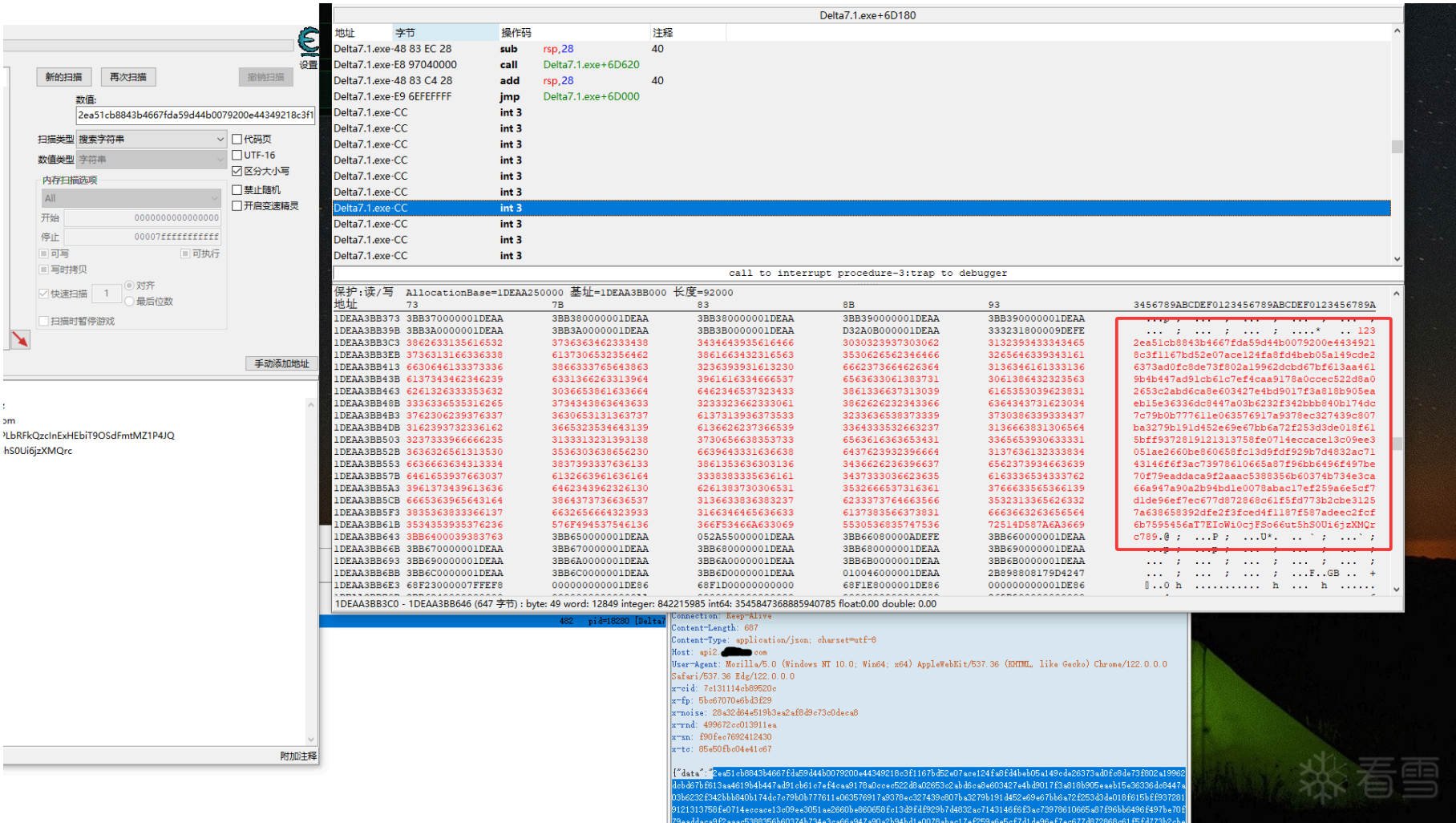
客户端验证的方式相同，问题就在于，客户端验证sign字段的方式我们可以在客户端那边搞到，服务器那边搞不到，怎么办呢？

先来试着找下客户端这边的验证字段吧，看看是不是固定的123456789吧...直接搜下行的密文数据，看内存里是否存在字符串拼接痕迹

搜了下，并没有....

搜上行数据看看呢...(也就是客户端发给服务器的密文数据)

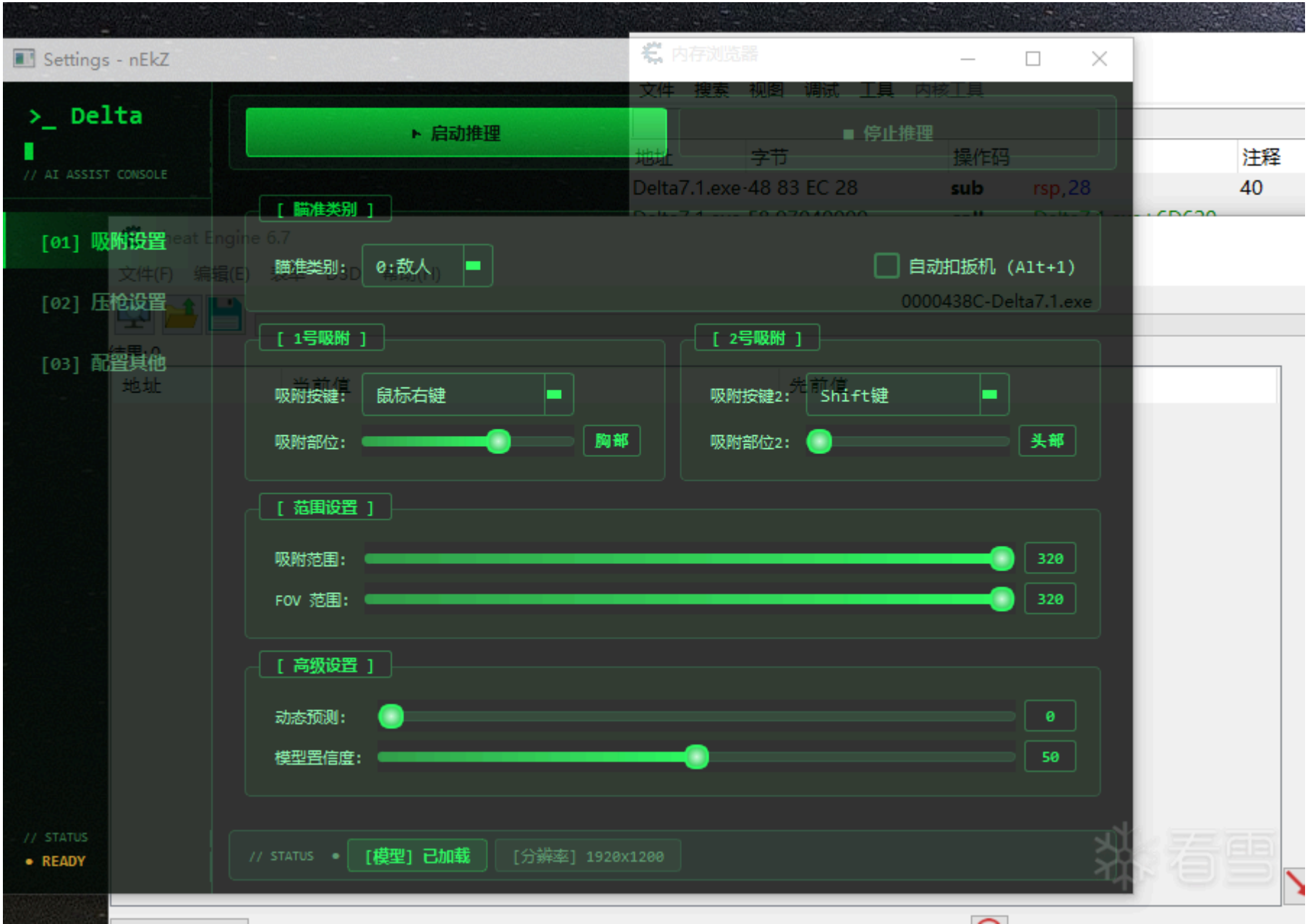
上行数据搜到了，并且自己把固定字符串和密文数据以及软件密钥拼接起来算了下md5是对的



啊？难道客户端不验证签名吗？

只是客户端算了一份sign发给服务器？

验证一下，直接把软件识别码、软件密钥、通讯密钥都改成我自己后台生成的然后自己生成卡密登录试下



....这作者是个人物，初始化、登录、版本号请求、心跳，包括登出，全部正常，所以客户端根本没验证这个sign签名字段...

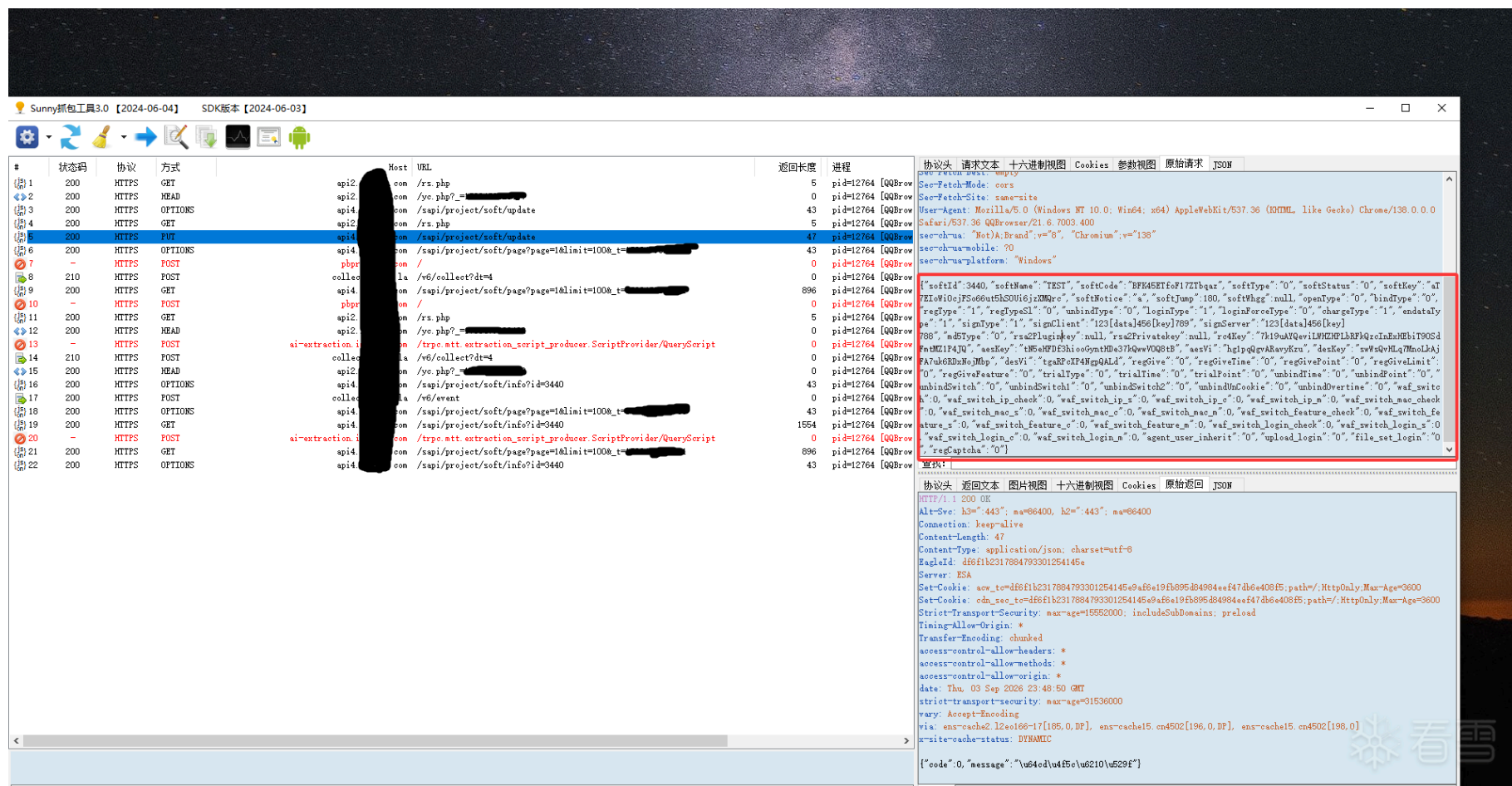
所以大概率客户端是没有校验sign的...那么如果客户端有sign的校验，而我们又拿不到服务端的固定用来算sign的那几个字符串，应该怎么办呢？

1、尝试用默认的123+密文的上行数据+456+软件密钥+789去算...试试看

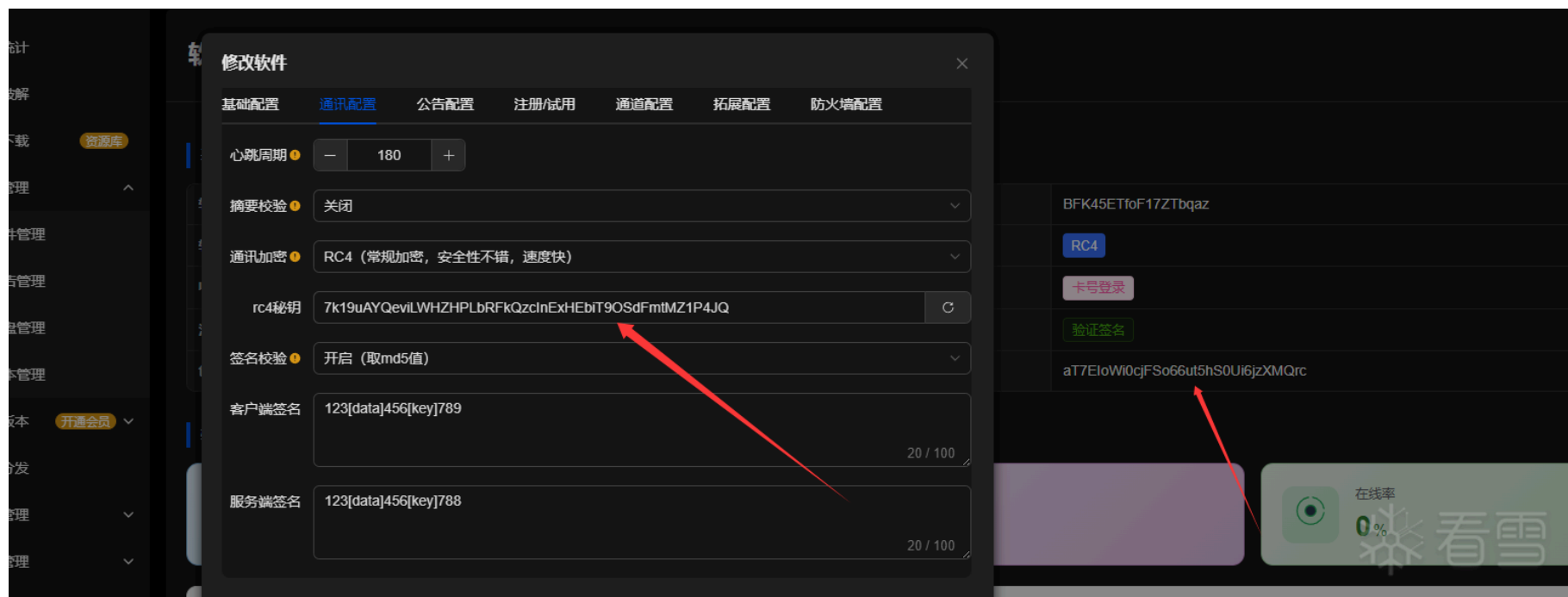
2、找它算MD5的函数，挂VEH让它返回任意的内容(如果是VMP，那么这个函数大概率是被VM的直接inlineHook是不行的)

3、如果客户端进行了校验，那么在客户端这边是绝对可以找到字符串的拼接痕迹的，直接搜密文的下行数据就行了，挨个看一下搜出来的每一个字符串的前后结尾处，就能看到它到底用了什么固定的拼接字符串

此外，在后台设置软件配置的时候(就是设置公告、软件密钥、通讯密钥那些东西的时候)发现，保存设置的时候，直接就是一个明目张胆的POST,也没有校验啥的....

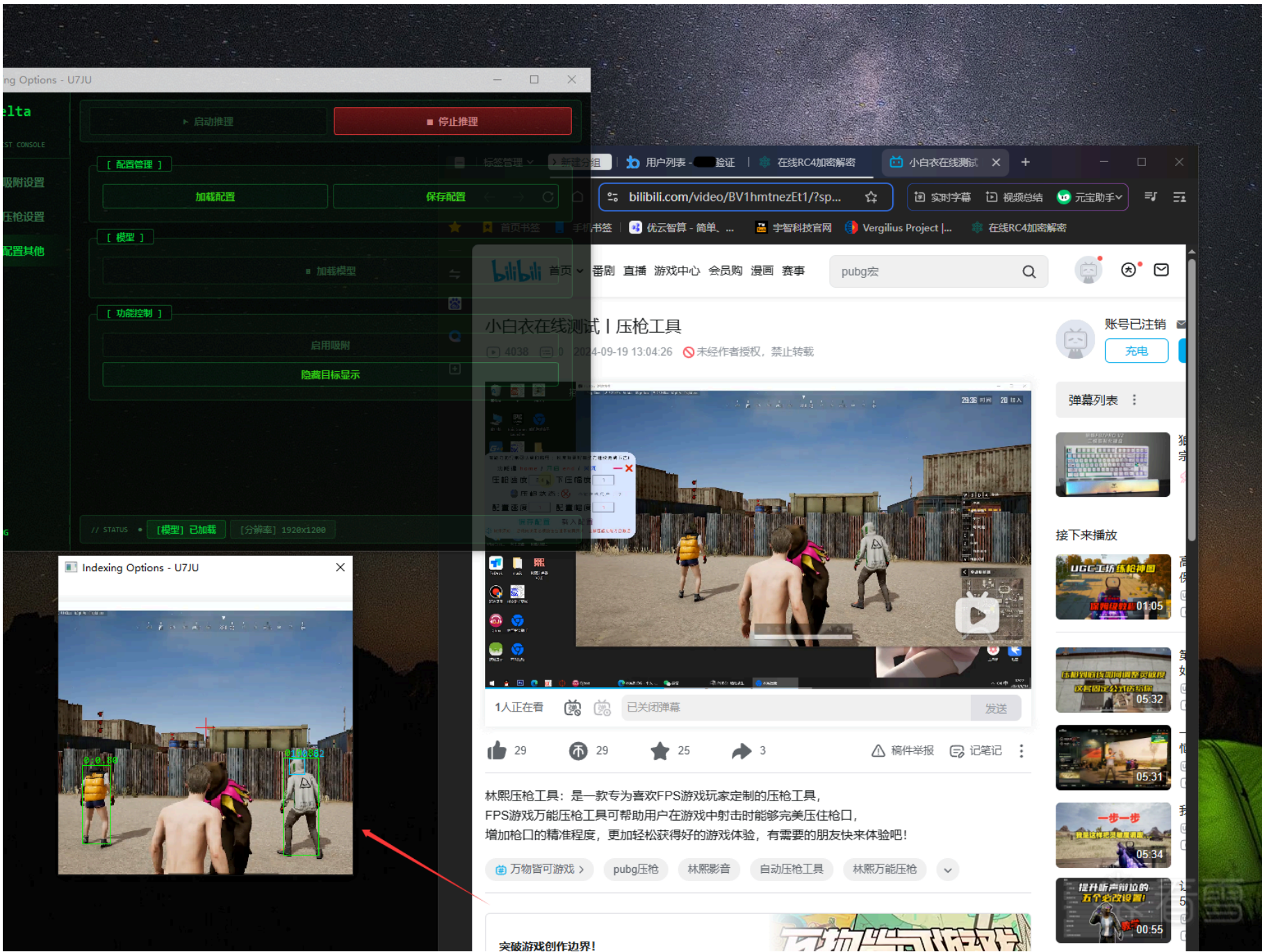


可以看到，请求体里面是应有尽有，这无疑使我们的"山寨"更加方便了....改请求体就行了，既然手动设置 软件识别码、软件密钥、通讯密钥这几个关键参数的时候不允许自定义，只允许用服务器下发的随机值，那么直接把请求体里面的软件密钥、通讯密钥改成目标软件的，那连写Patch的事都省了哈哈...不过软件识别码我认为不要改成目标软件的，因为如果服务器区分软件位时不看用户，只看软件识别码的话，软件识别码一有重复，恐怕作者会比他的服务器先炸掉...(这只是猜测，这带有攻击性质，因此不做测试，只修改软件密钥和通讯密钥，做"山寨"测试即可)



经过尝试，成功把软件密钥和通讯密钥改成了目标软件的，然后客户端这边只需要改一下软件识别码，改成自己软件位的就行了，补丁都懒得写了，CE直接改一下，验证效果就行了(其实上面已经验证过了)

随便找个PUBG的视频让它识别下，没问题



至于验证的开发者说的这段话



我们得思考一下，像这样的网络验证模式，把一些关键的变量放到云变量、云计算之类的一些东西里面，真的安全吗？其实不一定...如果是单纯的云变量，拿到通讯密钥以后，抓请求云变量的包，然后拿密钥解一下就出来了，云变量一般情况下是周期性变动的，比如游戏的基址之类的，变动得不会特别频繁那么如果是云变量呢？一般的云变量其实就是拿一些客户端那边的信息，再加上一些....意想不到的信息，算出一个相对用户或者相对进程的动态值，然后下发给客户端，客户端再去验证，增加破解成本这块，还是云计算比较有效，不过效果也有限，毕竟依赖客户端的因素嘛

希望这篇对正在学习这方面的人有帮助，主要是这个网络验证的开发者,声明里写着防止域名劫持...作为一个收费的网络验证，我觉得不应该把这件事推到消费者身上去，因为你们稍微做些相应的措施，比让消费者自己去防范某些东西效率要来的高得多，比如后台可以搞点加密、执行必要操作后立刻把内存里的残留和字符串拼接痕迹相关的内存free掉等等，不要让用户自己选择性地做这些必要的事情,这样或许口碑会好很多...


收藏 · 3


免费 · 3


打赏


分享

赞赏记录

参与人	雪币	留言	时间
我的小拇指啊		你的分享对大家帮助很大，非常感谢！	3天前
cloudhoter		谢谢你的细致分析，受益匪浅！	3天前
日暮归途		非常支持你的观点！	3天前

最新回复 (0)



Baiusc

内容

温馨提示：点击 引用 按钮再回帖，可回复并 短消息 提示对方

回帖

表情

高级回复 MarkDown / 富文本

返回